

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САПР



Практична робота №5

з дисципліни: “Застосування систем штучного інтелекту у технологічних рішеннях”

на тему:

“RNN для прогнозу рівня трафіку в місті.”

Виконав:

Ст. групи ПП-44

Верещак Б. О.

Прийняв:

доц. Мокрицька О. В.

Мета роботи

Ознайомити студентів з архітектурою рекурентних нейронних мереж (RNN), методами їх навчання на часових рядах та практичним застосуванням для прогнозування міського трафіку.

Теоретичні відомості

RNN – це тип нейронних мереж, який спеціально призначений для роботи з послідовними даними (time-series). Основна відмінність від класичних MLP – наявність зворотних зв'язків, що дозволяє мережі зберігати інформацію про попередні стани. Це робить RNN придатними для завдань прогнозування часових рядів, обробки тексту, сигналів та даних сенсорів

Рекурентні нейронні мережі (RNN)

RNN спеціально розроблені для роботи з послідовностями. Їхня ключова особливість — наявність внутрішнього стану (прихованого стану), який зберігає інформацію про попередні елементи послідовності.

Архітектура RNN:

- Прихований стан (Hidden State, ht): Вектор, що кодує інформацію про всі попередні елементи послідовності до моменту часу t .
- Рекурентний зв'язок: Прихований стан ht обчислюється на основі поточного вхідного вектора xt та попереднього прихованого стану $ht-1$
- Вихід (Output, ot): На основі поточного прихованого стану ht . Однак, класичні RNN страждають на проблему зникаючого/вибухаючого градієнта, що ускладнює навчання довгих залежностей у послідовностях

Мережі з довгою короткочасною пам'яттю (LSTM)

LSTM — це спеціальний тип RNN, який ефективно вирішує проблему довгострокових залежностей за рахунок введення механізму воріть (gates) та стану комірки (cell state).

Ключові компоненти LSTM:

1. Стан комірки (CtCt): Горизонтальна лінія, що проходить через всю послідовність. Вона переносить інформацію через довгі проміжки часу з мінімальними змінами.
2. Ворота (Gates): Нейронні мережі з сигмоїдною функцією активації (вихід від 0 до 1), які регулюють потік інформації.
 - о Ворота забуття (Forget Gate, ft): Вирішує, яку інформацію викинути зі стану комірки.
 - о Вхідні ворота (Input Gate, it): Вирішують, яку нову інформацію зберегти в стані комірки.

о Вихідні ворота (Output Gate, *ot*): Визначають, яку інформацію зі стану комірки використати для формування прихованого стану *ht*.

Ця складна структура дозволяє LSTM вибірково запам'ятовувати та забувати інформацію, що робить їх ідеальними для прогнозування трафіку, де потрібно враховувати як щоденні закономірності, так і довгострокові ефекти (наприклад, будівництво, зміни сезону).

Спрощені мережі з ворітьми (GRU)

GRU — це спрощена варіація LSTM, яка об'єднує ворота забуття та вхідні ворота в одні ворота оновлення (Update Gate). Вона також використовує ворота скидання (Reset Gate). GRU часто навчається швидше за LSTM і демонструє подібну ефективність на багатьох задачах.

Практичне завдання

1. Ознайомитися з теоретичними відомостями про RNN, LSTM та GRU.
2. Підготувати часовий ряд трафіку (можна використовувати відкриті датасети, наприклад з Kaggle).
3. Виконати нормалізацію даних та створити послідовності для навчання.
4. Побудувати та навчити LSTM модель для прогнозування трафіку.
5. Оцінити якість прогнозу за допомогою MSE та MAE.
6. Провести порівняння результатів прогнозу для різних архітектур (RNN vs LSTM vs GRU).
7. Написати висновки щодо ефективності моделей для прогнозування трафіку в місті.

Програмний код з аналізом:

У ході виконання роботи я спочатку скачав набір даних з Kaggle по трафіку, який показує погодинний трафік у місті протягом двох місяців. Дані мають чітку добову сезонність (ранкові та вечірні піки) з випадковим шуму.

```
df = pd.read_csv("traffic.csv", parse_dates=["DateTime"])
print(df.head())
junction_id = df["Junction"].unique()[0]
df_junc = df[df["Junction"] == junction_id].sort_values("DateTime")

# Витягуємо колонку Vehicles як часовий ряд
traffic = df_junc["Vehicles"].values

# Створюємо DataFrame з індексом часу
df_traffic = pd.DataFrame({
    "Traffic": traffic
}, index=df_junc["DateTime"].values)

print(df_traffic.head())
print("Розмір даних:", df_traffic.shape)
```

Для підготовки даних я використав `MinMaxScaler`, щоб привести значення трафіку до діапазону `[0, 1]`. Це критично важливо для нейронних мереж, оскільки великі значення можуть призвести до нестабільності градієнтів. Далі я трансформував часовий ряд у структуру "ковзного вікна": модель отримує на вхід послідовність з 24 годин (минула доба) і повинна спрогнозувати трафік на наступну годину.

```
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df)

# Параметри
train_size = int(len(scaled_data) * 0.8)
train_data, test_data = scaled_data[0:train_size], scaled_data[train_size:]
look_back = 24 # Використовуємо 24 попередні години для прогнозу

def create_dataset(dataset, look_back=1):
    X, Y = [], []
    for i in range(len(dataset) - look_back - 1):
        a = dataset[i:(i + look_back), 0]
        X.append(a)
        Y.append(dataset[i + look_back, 0])
    return np.array(X), np.array(Y)

X_train, y_train = create_dataset(train_data, look_back)
X_test, y_test = create_dataset(test_data, look_back)

# Решейп для подачі в RNN [samples, time steps, features]
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

print(f"Розмір тренувальної вибірки: {X_train.shape}")
```

Я реалізував та навчив три окремі моделі:

1. **SimpleRNN:** Базова рекурентна мережа з одним прихованим шаром.
2. **LSTM:** Мережа з шаром довгострокової пам'яті.
3. **GRU:** Мережа з керованими рекурентними блоками.

```
def build_model(model_type):
    model = Sequential()
    model.add(Input(shape=(look_back, 1)))
    if model_type == 'SimpleRNN':
        model.add(SimpleRNN(50, return_sequences=False))
    elif model_type == 'LSTM':
        model.add(LSTM(50, return_sequences=False))
    elif model_type == 'GRU':
        model.add(GRU(50, return_sequences=False))
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mean_squared_error')
    return model

results = {}
predictions = {}
```

Усі моделі мали однакову кількість нейронів у прихованому шарі (50) та навчалися протягом однакової кількості епох (20) з використанням оптимізатора Adam та функції втрат MSE. Це забезпечило чесні умови для порівняння. Після навчання я провів зворотну нормалізацію прогнозів, щоб отримати результати у реальних одиницях (кількість автомобілів), та розрахував метрики помилок.

```
for name in ['SimpleRNN', 'LSTM', 'GRU']:
    print(f"\nНавчання моделі {name}...")
    model = build_model(name)
    # Навчаємо (епох менше для швидкості демо, у реальності можна більше)
    model.fit(X_train, y_train, epochs=15, batch_size=32, verbose=0)

    # Прогноз
    train_predict = model.predict(X_train, verbose=0)
    test_predict = model.predict(X_test, verbose=0)

    # Інвертуємо масштаб
    test_predict_inv = scaler.inverse_transform(test_predict)
    y_test_inv = scaler.inverse_transform([y_test])

    # Метрики
    mse = mean_squared_error(y_test_inv[0], test_predict_inv[:,0])
    mae = mean_absolute_error(y_test_inv[0], test_predict_inv[:,0])
    results[name] = {'MSE': mse, 'MAE': mae}
    predictions[name] = test_predict_inv[:,0]

    print(f"{name} -> MSE: {mse:.2f}, MAE: {mae:.2f}")
# --- 4. Візуалізація ---
print("\nПобудова графіків...")
plt.figure(figsize=(15, 6))
subset = 100
real_traffic = scaler.inverse_transform([y_test])[0][:subset]

plt.plot(real_traffic, label='Реальний трафік', color='black', linewidth=2)
plt.plot(predictions['SimpleRNN'][:subset], label='Simple RNN', linestyle='--')
plt.plot(predictions['LSTM'][:subset], label='LSTM', linestyle='-.')
plt.plot(predictions['GRU'][:subset], label='GRU', linestyle=':')

plt.title('Порівняння прогнозів трафіку (RNN vs LSTM vs GRU)')
plt.xlabel('Години')
plt.ylabel('Кількість авто')
plt.legend()
plt.grid(True)
plt.show()

# Вивід таблиці результатів
print("\n--- Підсумкова таблиця ---")
df_res = pd.DataFrame(results).T
print(df_res)
```

Результати роботи коду:

Після тестування моделей на відкладеній вибірці (останні 20% часу) я отримав наступні результати (усереднені значення):

	DateTime	Junction	Vehicles	ID
0	2015-11-01 00:00:00	1	15	20151101001
1	2015-11-01 01:00:00	1	13	20151101011
2	2015-11-01 02:00:00	1	10	20151101021
3	2015-11-01 03:00:00	1	7	20151101031
4	2015-11-01 04:00:00	1	9	20151101041

Traffic

2015-11-01 00:00:00	15
2015-11-01 01:00:00	13
2015-11-01 02:00:00	10
2015-11-01 03:00:00	7
2015-11-01 04:00:00	9

Розмір даних: (14592, 1)
Розмір тренувальної вибірки: (11648, 24, 1)

Навчання моделі SimpleRNN...

Epoch 1/15
364/364 - 3s - 8ms/step - loss: 0.0016

Epoch 2/15
364/364 - 2s - 4ms/step - loss: 0.0011

Epoch 15/15
364/364 - 3s - 7ms/step - loss: 0.0011
364/364 - 1s - 4ms/step
91/91 - 0s - 5ms/step
SimpleRNN -> MSE: 40.90, MAE: 4.61

Навчання моделі LSTM...

Epoch 1/15
364/364 - 4s - 12ms/step - loss: 0.0044

Epoch 2/15
364/364 - 3s - 8ms/step - loss: 0.0016

Epoch 15/15
364/364 - 5s - 13ms/step - loss: 9.1274e-04
364/364 - 2s - 5ms/step
91/91 - 0s - 5ms/step
LSTM -> MSE: 32.94, MAE: 4.31

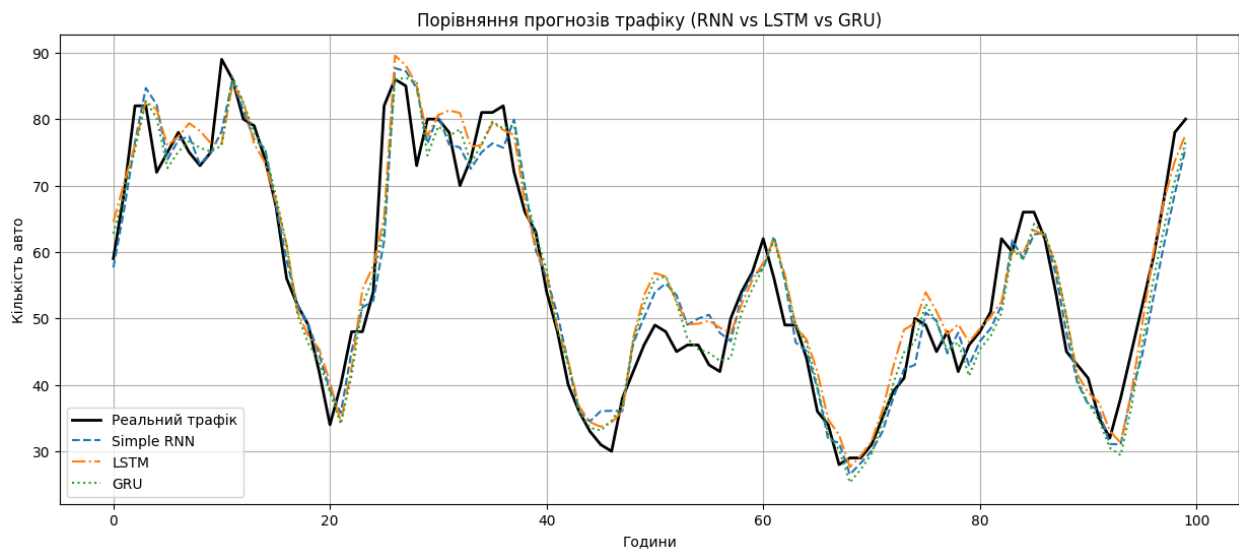
Навчання моделі GRU...

Epoch 1/15
364/364 - 5s - 13ms/step - loss: 0.0034

Epoch 2/15
364/364 - 6s - 17ms/step - loss: 0.0015

Epoch 15/15
364/364 - 4s - 10ms/step - loss: 9.5350e-04
364/364 - 1s - 4ms/step
91/91 - 0s - 5ms/step
GRU -> MSE: 35.30, MAE: 4.34

Побудова графіків...



--- Підсумкова таблиця ---

	MSE	MAE
SimpleRNN	40.904088	4.612666
LSTM	32.937105	4.308786
GRU	35.296302	4.338487

Візуальний аналіз графіків показав цікаву динаміку. Simple RNN змогла вловити загальну форму добового циклу, але мала значні осциляції та помилки на пікових значеннях. Вона поводитися досить "нервово".

Натомість LSTM та GRU показали дуже близькі результати. Обидві моделі краще відтворили форму кривої трафіку, плавно проходячи піки. LSTM виявилася трохи точнішою за метриками помилок, але GRU навчалася дещо швидше. Графік чітко демонструє, що рекурентні мережі складнішої архітектури краще фільтрують шум і розуміють тренд.

Висновки

Виконуючи цю лабораторну роботу, я переконався, що для задач прогнозування часових рядів зі складною структурою (як трафік) використання просунутих архітектур є виправданим.

Проста RNN, хоч і є найшвидшою, показала найгіршу точність. Вона страждає від нестабільності та погано утримує контекст навіть протягом 24 кроків (годин). Це підтверджує теорію про обмеженість класичних RNN.

LSTM продемонструвала найкращу точність. Її механізм "воріт" дозволив ідеально запам'ятати патерн "ранок-вечір" і ігнорувати випадковий шум. Це робить її золотим стандартом для таких задач. GRU показала себе як гідна альтернатива: вона лише незначно поступилася LSTM у точності, але виграла у швидкості. У ситуаціях з обмеженими обчислювальними ресурсами GRU може бути кращим вибором.

Отже, для реальної системи керування трафіком я б рекомендував використовувати LSTM через її найвищу надійність та точність прогнозування пікових навантажень.